DIGITAL LOGIC DESIGN

(CE_118)

Hardware Description Language

Verilog

Objective

- Describe what is the lexical convention
- Describe the data types allowed in Verilog
- Describe the module port rule allowed in Verilog
- Describe what is the dataflow modeling
- Describe how to use continuous assignments
- Describe how to use procedural assignments
- Describe the operation of the operators used in Verilog
- Describe the operands may be used associated with a specified operator

Content

- Lexical convention
- Data Types
- Module Port Rule
- Dataflow modeling
- Expression, operator, operand
- Continuous assignment
- Procedural assignment

How to represent Hardware?

- **Classical design method**

  - Schematic

  - Hand-draw or machine-draw

- **Computer-based language method**

  - Hardware Description Language (HDL): **Verilog**, **VHDL**

  - Fast, popularly used to design complex circuit with large and very large scale

```
module Flop (reset, din, clk, qout);
    input reset, din, clk;
    output qout;
    reg qout;
    always @ (negedge clk) begin
        if (reset) qout <= #8 1'b0;
        else qout <= #8 din;
    end
endmodule
```

Module

- **Module** is the basic building block in Verilog

  - Can be an element or a collection of lower-level design blocks

  - Provide functionality for higher-level block through its port interface

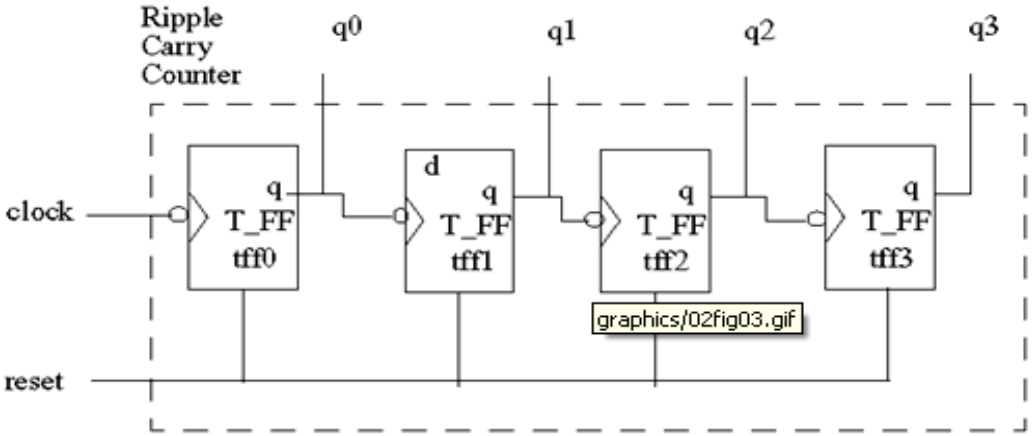
  - Hide internal implementation

  - Is used at many places in the design

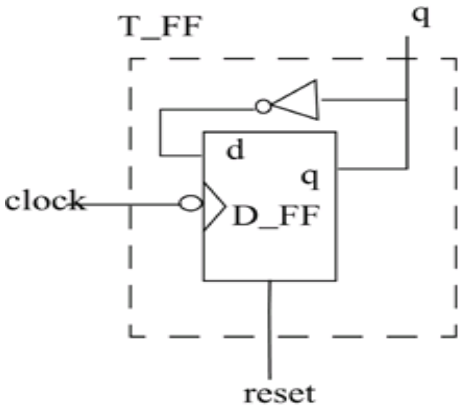
  - Allows designers modify module internals without effecting the rest of design

Module

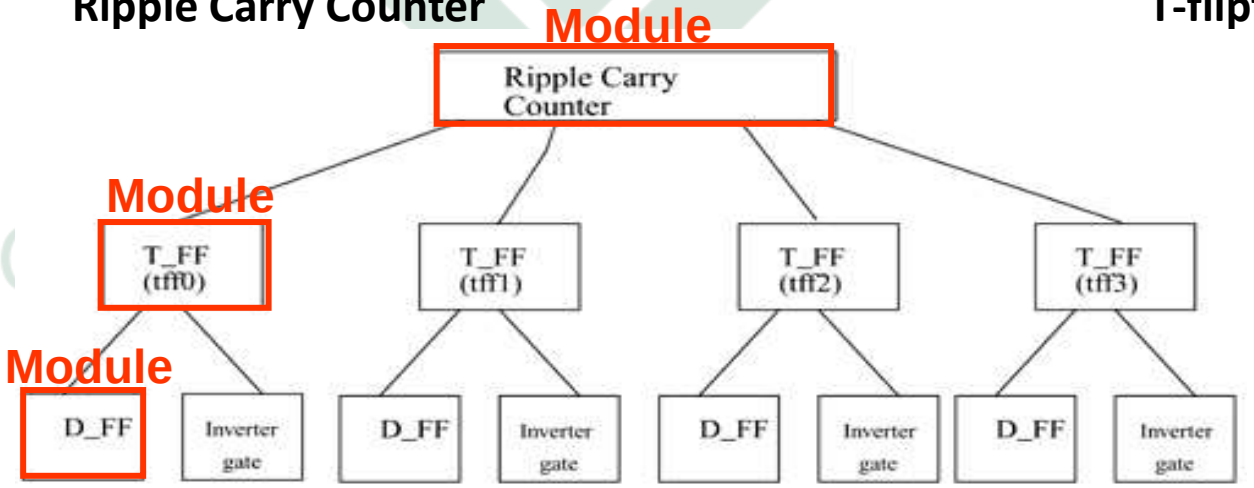
Example: 4-bit Ripple Carry Counter



Ripple Carry Counter



T-flipflop



Design Hierarchy

Module

- **Module description**

module *module name* **(** *port name*, *port name*,…**);**

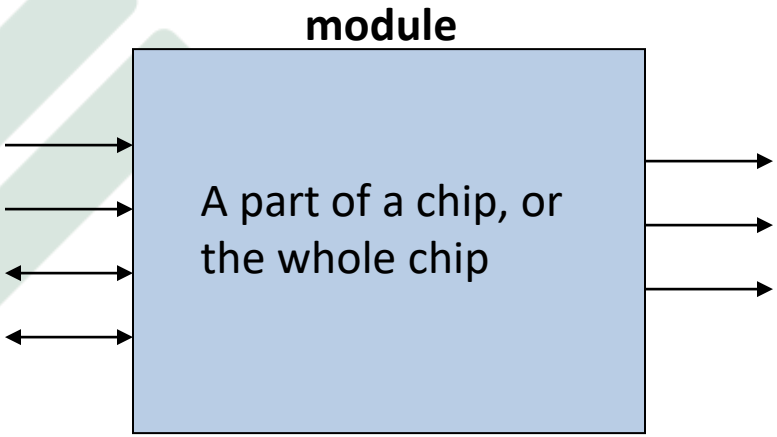
module_port declaration

data type declaration

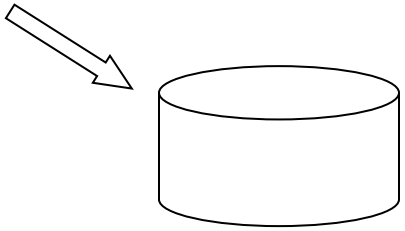
logic description part

endmodule

A module definition



The file name for RTL source must be
"*module name.v*"



Lexical convention

- Operator
- White space
- Comment
- Number
- String
- Identifier
- Keyword

Lexical convention

- Verilog is a case-sensitive language.
- Whitespace: **\b** blank, **\t** tab, **\n** new line
- Comment: **/*..*/** or **//……..**
- Operator: unary, binary, ternary

  a = ~ b; // ~ is a unary operator. b is the operand

  a = b && c; // && is a binary operator. b and c are operands

  a = b ? c : d; // ?: is a ternary operator. b, c and d are operands

- Keywords: **module**, **end module**, **input, output, inout**, **wire, reg,** …

Lexical convention

- **Number**

  - Two forms to express numbers:

    * 37 : 32 bit decimal 37, or
    * **<size>'<base_format><number>**

  Ex:

10'hFA	10 bits hexadecimal number FA (00_1111_1010)
1'b0	1 bit binary number 0 (0)
6'd30	6 bits decimal number (011110), decimal 30
15'o10752	15 bits octal number (001,000,111,101,010), decimal 4586
	4'b0 is equal to 4'b0000
4'b1	is equal to 4'b0001
4'bz	is equal to 4'bzzzz
4'bx	is equal to 4'bxxxx
-8 'd 6	The two's complement of 6, held in 8 bits

Lexical convention

- **Strings** are stored as a sequence of 8 bit ASCII values
- Strings are variables of ***reg*** type

```
module string_test;
reg [8*14:1] stringvar;
initial begin
    stringvar = "Hello world";
    $display("%s is stored as %h", stringvar,stringvar);
    stringvar = {stringvar,"!!!"};
    $display("%s is stored as %h", stringvar,stringvar);
end
endmodule
```

The output is as follows:

```
Hello world is stored as 00000048656c6c6f20776f726c64
Hello world!!! is stored as 48656c6c6f20776f726c64212121
```

Lexical convention

- **Identifier**:
  - specified by a letter or underscore followed by more letter or digits, or signs ($, _).
  - The first character must NOT be a digit or $; it can be a letter or an underscore
  - Identifiers are case sensitive.
  - Identifier can up to 1024 characters
  - Example:
    - shiftreg_a
    - busa_index
    - error_condition
    - merge_ab
    - _bus3
    - n$657

Data Types

- **Value set**:
  - 0 - represents a logic zero, or false condition
  - 1 - represents a logic one, or true condition
  - x - represents an unknown logic value
  - z - represents a high-impedance state
- **Net**
- **Variable**
- **Vector**
- **Array**
- **Memory**
- **Parameter**

Data types

2 main groups: **Net** and **Variable**

- **Net** represents physical connections between structural entities, such as gates.
  - **Net** does not store a value, except "trireg" net
  - Default value is z (trireg default value is x)
  - Net types:

wire	tri	tri0	supply0
wand	triand	tri1	supply1
wor	trior	trireg	uwire

(Ref. Verilog IEEE book for detail of net types)

Ex:

- **wire** w1, w2; //declare 2 wires
- wand w;

Data types

- **Variable**: hold a value until another value assigned to it

- Used in behavioral description (procedural assignment)

  reg : unsigned integer variables of varying bit width

  integer : 32-bit signed integer

  time : 64-bit unsigned integer

  real : signed floating-point

  realtime: store time as a real value

- **reg**, **time**, **integer** (default value is x),

  real , **realtime** (default value is 0)

Data Types

- Registers
  - an abstraction of a data storage element
  - keyword for the register data type is **reg**.
  - A register stores a value from one assignment to the next.
  - The default initialization value for a reg data type is the unknown value, x.
  - Example:

```
reg reset;
initial begin
        reset = 1'b1;
        #100 reset = 1'b0;
end
```

Data types

- **Vector**: Multiple ***net*** or ***reg*** data types are declared by specifying a range, is known as a vector

```
wand w;                          // a scalar net of type "wand"
tri [15:0] busa;                 // a three-state 16-bit bus
trireg (small) storeit;          // a charge storage node of strength small
reg a;                           // a scalar reg
reg[3:0] v;                      // a 4-bit vector reg made up of (from most to
                                 // least significant)v[3], v[2], v[1], and v[0]
reg signed [3:0] signed_reg;        // a 4-bit vector in range -8 to 7
reg [-1:4] b;                    // a 6-bit vector reg
wire w1, w2;                     // declares two wires
reg [4:0] x, y, z;               // declares three 5-bit regs
```

Data types

- **Array** declaration for a net or variable that is either scalar or vector

Declaration	Element type
reg x[11:0];	scalar **reg**
wire [0:7] y[5:0];	8-bit-wide vector wire indexed from 0 to 7
reg [31:0] x [127:0];	32-bit-wide **reg**

- **Memory**: is one-dimension array with element of type **reg**. These memories can be used to model ROM, RAM, or reg files

Example:

```
reg mem1bit [0:1023];
reg [16:0] membyte [0:1023];
assign rdata = membyte[969];
```

0	1	2	3	4	5	6	7
1							
2							
3							

Data Types

- **Parameters**
  - Verilog parameters do not belong to either the net or variable group, they are constants
  - Syntax:
    - <parameter_declaration> = parameter <list_of_assignments>;
  - Example:

```
parameter msb = 7; // defines msb as a constant value 7
parameter e = 25, f = 9;// defines two constant numbers
parameter average_delay = (r + f) / 2;
parameter byte_size = 8, byte_mask = byte_size - 1;
parameter r = 5.7; //declares r as a ’ real’ parameter
```

Expression, operator, operand

- expression = operators + operands
  - Operators
  - Operands

Expression: Operands

- Constant number or string
- Parameter
- Net
- Variable (reg, integer, time, real, realtime)
- Array element
- Bit-select or part-select (not for real, realtime)
- Function call that returns any of the above

Constant

Data types

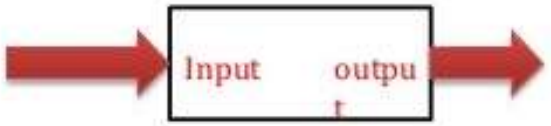
Expression: Operators

Arithmetic	+ - * / % **
Logical	! && ||
Logical equality	== !=
Case equality	=== !===
Bitwise	~ & | ^ ^~ (or ~^)
Relational	< > >= <=
Unary reduction	& ~& | ~| ^ ^~ (or ~^)
Shift	<< >> <<< >>>
Concatenation	{}
Replication	{{}}
Condition	?:
Unary	+ -

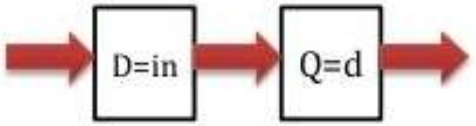
Ref. book for detail of each operator!

Level of modeling

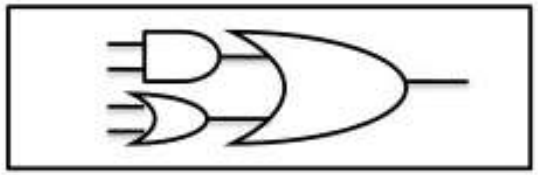
- Behavioral Level



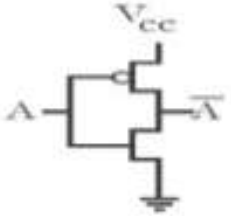
- Data flow Level



- Gate level



- Switch level



- There are different ways of modeling a hardware design. Choose an appropriate model to design Combinational or Sequential Circuit.
- Some books do not classify Dataflow modeling as a separate modeling type.

Structural model

- **Structural**

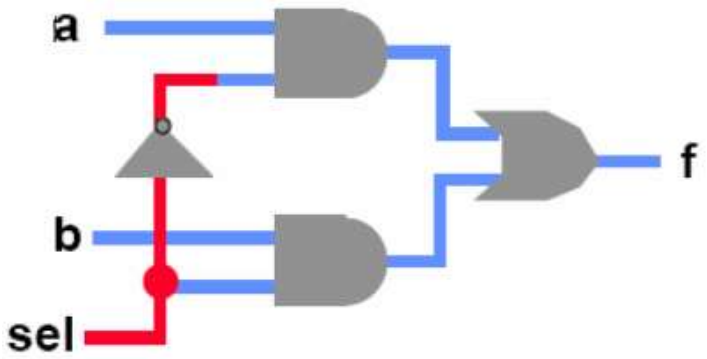
- Explicit structure of the circuit
- How a module is composed as an interconnection of more primitive modules or components
- E.g. Each logic gate initially instantiated and connected to others

- **In Verilog**, a structural model consists of:

- List of connected components
- Like schematics, but using **text**: **netlist**
- Boring when write, and hard to decode
- Essential without integrated design tools

Structural model

- Structural Models are built from gate primitives, switches, and other modules
- Describe the logic circuit using logic gates




```
module mux (f, a, b, sel);
      output   f;
      input    a, b, sel;

      and #5   g1 (f1, a, nsel),
               g2 (f2, b, sel);
      or   #5  g3 (f, f1, f2);
      not      g4 (nsel, sel);
endmodule
```

Structural model

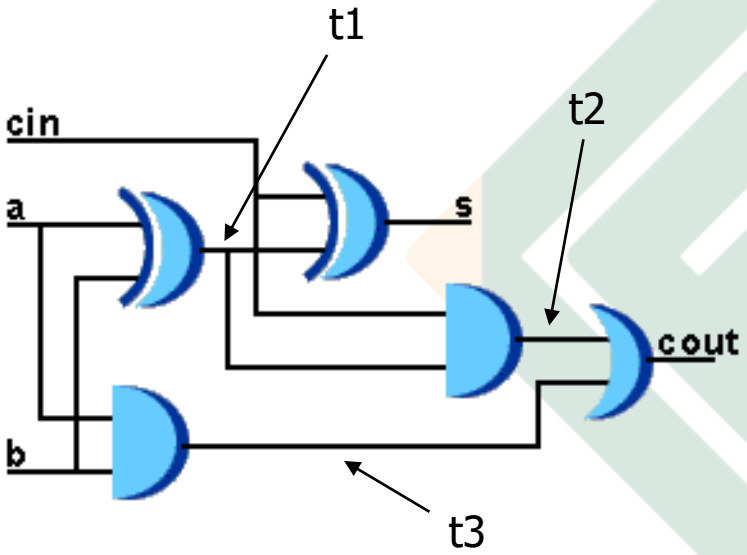
- Primitive gates:
  - 12 primitive logic gates predefined in the Verilog HDL

and	nand	logical AND/NAND
or	nor	logical OR/NOR
xor	xnor	logical XOR/XNOR
buf	not	buffer/inverter
bufif0	notif0	Tristate with low enable
bifif1	notif1	Tristate with high enable

  - **Advantanges:**
    - ✓ Gates provide a much closer one-to-one mapping between the actual circuit and the model.
    - ✓There is no continuous assignment equivalent to the bidirectional transfer gate.

Structural model

Example:



Full adder 1-bit

```
// Define a 1-bit full adder
module fulladd(sum, c_out, a, b, c_in);

// I/O port declarations
output sum, c_out;
input a, b, c_in;

// Internal nets
wire s1, c1, c2;

// Instantiate logic gate primitives
xor (s1, a, b);
and (c1, a, b);
xor (sum, s1, c_in);
and (c2, s1, c_in);
xor (c_out, c2, c1);
endmodule
```

Behavioral model

- **What is Behavioral model ?**
  - More like a procedure in a programming language, but NOT.
  - Program describes input/output behavior of circuit, tell what you want to have happen, NOT what gates to connect to make it happen.
  - Describe what a component does, not how it does
  - Many structural models could have the same behavior (different implementations of one expression)
  - Synthesized into a circuit that has this behavior
  - Result is only as good as the tools

Behavioral model

- **Why behavioral model?**
  - Good for more abstract models of circuits
  - Easier to write
  - More flexible
  - Provide sequencing
  - A much easier way to write testbenches
  - Allow both model and the testbench to be described together

Assignments

- The continuous assignment
  - assigns values to *nets*
- The procedural assignment
  - assigns values to *registers*

Statement type	Left-hand side
continuous assignment	net (vector or scalar) constant bit-select of a vector net constant part-select of a vector net concatenation of any of the above 3
procedural assignment	register (vector or scalar) bit-select of a vector register constant part-select of a vector register memory element concatenation of any of the above 4

Continuous assignment

- Drive a value onto a net

  assign out = i1 & i2; //out is net; i1 and i2 are nets

Left-hand side	Right-hand side
Net (vector or scalar) Bit-select or part-select of a vetor net Concatenation of any of the above	Net, register, function call (any expression that gives a value)

- Always active
- Delay value: control time when the net is assigned value

  assign #10 out = in1 & in2; //delay of performing computation,
  //only used by simulator, not synthesis

Continuous assignment

Examples:

wire out = in1 & in2; *//scalar net*

//implicit continuous assignment, declared only once

assign addr[15:0] = addr1_bits[15:0] ^ addr2_bits[15:0]; *//vector net*

assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in; *//concatenation*

```
module adder (sum, carry_out, carry_in, ina, inb);
output [3:0] sum;
output carry_out;
input [3:0] ina, inb;
input carry_in;
wire carry_out, carry_in;
wire [3:0] sum, ina, inb;

assign {carry_out, sum} = ina + inb + carry_in;
endmodule
```

Continuous assignment

- Because the assignment is done always, exchanging the written order of the lines of continuous assignments has no influence on the logic
- Common error

  - Not assigning a wire a value

  - Assigning a wire a value more than one

- Target (LHS) is NEVER a **reg** variable

  (LHS: Left-Hand Side)

Procedural Assignment

- Assign value to variable data types

Left-hand side	Right-hand side
- reg, integer, time, real, realtime - Bit-select, part-select of **reg, integer**, **time** - Memory word - Concatenation of any of the above	Net, variable, function call (Any expression that evaluates a value)

- Occur in procedures: **initial**, **always, task,** and **function**
- Being thought of as "triggered" assignment
- Variable hold value until the next assignment

Examples:


```
reg[3:0] a = 4'h4;

//This is equivalent to writing
reg[3:0] a;
initial a = 4'h4;
```

Procedural assignment

- *Difference between Continuous and Procedural assignment:*

- **Continuous assignments** drive **nets**, nets are evaluated and updated whenever an input operand changes value.

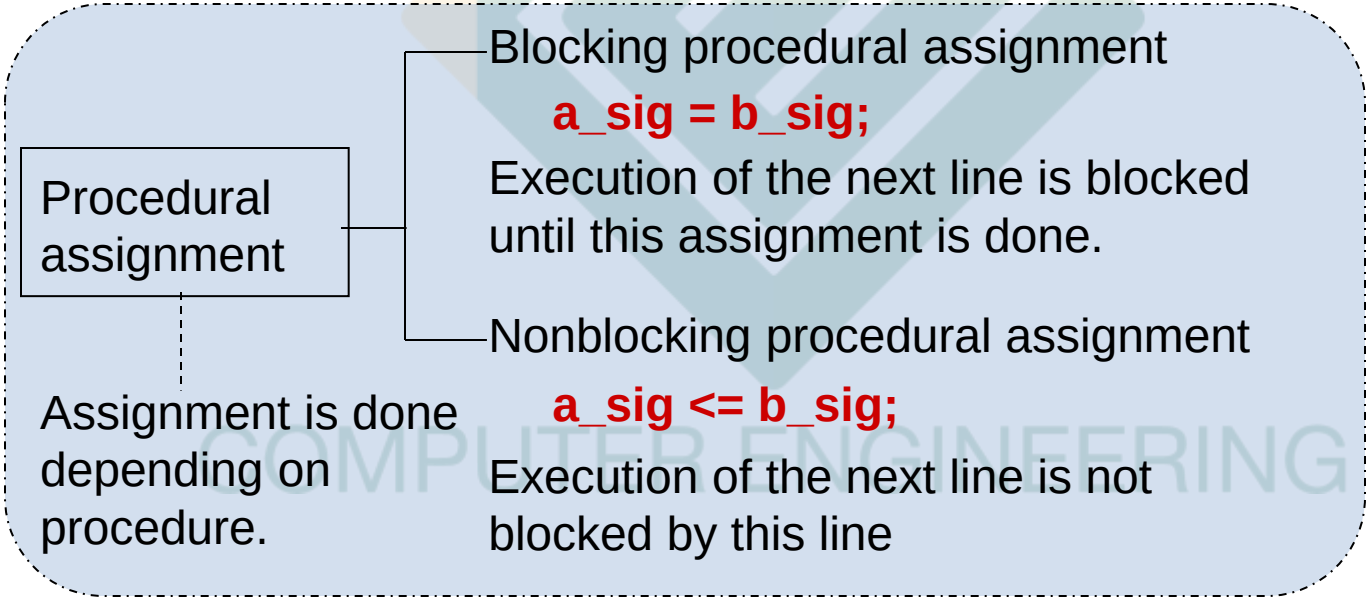
- **Procedural assignments** update the value of **variables** under the control of the procedural flow constructs that surround them

Procedural assignment

- 2 types of procedural assignment:

  blocking and non-blocking procedural assignment

  -> specify different flow in a sequential block



Procedural assignment

- **Example**

```
//non_block1.v
module non_block1;
reg a, b, c, d, e, f;

//blocking assignments
initial begin
    a = #10 1; // a will be assigned 1 at time 10
    b = #2 0;  // b will be assigned 0 at time 12
    c = #4 1;  // c will be assigned 1 at time 16
end
//non-blocking assignments
initial begin
    d <= #10 1; // d will be assigned 1 at time 10
    e <= #2 0;  // e will be assigned 0 at time 2
    f <= #4 1;  // f will be assigned 1 at time 4
end
endmodule
```

Initial & Always

- Two basic components of behavioral modeling: initial, always construct.
- Each initial and always construct starts a separate activity flow in Verilog.
- initial and always can not be nested.
- In the code: **initial** and **always** start together at simulation time 0, initial executes once, always executes repetitively.
- They contain behavioral statements like: procedural assignment, if…else, case, loop statements.

```
module behave;
reg [1:0] a, b;

initial begin
    a = 'b1;
    b = 'b0;
end
always begin
    #50 a = ~a;
end
always begin
    #100 b = ~b;
end

endmodule
```

Initial

- Variable and Net can be initialized when they are declared

```
module adder (sum, co, a, b, ci);

output reg [7:0] sum = 0; //Initialize 8 bit output sum
output reg co = 0; //Initialize 1 bit output co
input [7:0] a, b;
input ci;
…..
endmodule
```

```
module adder (output reg [7:0] sum = 0,
              output reg co = 0,
              input [7:0] a, b,
              input ci );
//it is illegal to redeclare any ports of the module
//in the body of the module
…
endmodule
```

Always

- **always** may be followed by timing control expression (@, #, **wait** -> see later)
- Statement following **always** is executed repeatedly until simulation stops. **$stop** or **$finish** to halt the simulation

```
module clock_gen (output reg clock);
//Initialize clock at time zero
initial
    clock = 1'b0;
//Toggle clock every half-cycle (time period = 20)
always
    #10 clock = ~clock;
initial
    #1000 $finish;
endmodule
```

Procedural timing control

- How does the behavioral model advance time?
  - **Delay control #**: delay a specific amount of time
  - **Event control @**: delay until an event occurs ("posedge", "negedge", or any change)

    - edge-sensitive timing control
  - **Statement wait**: delay until an event occur

    - level-sensitive timing control
- In Verilog, if there are no timing control statements, the simulation time does not advance.

Procedural timing control

- Examples:

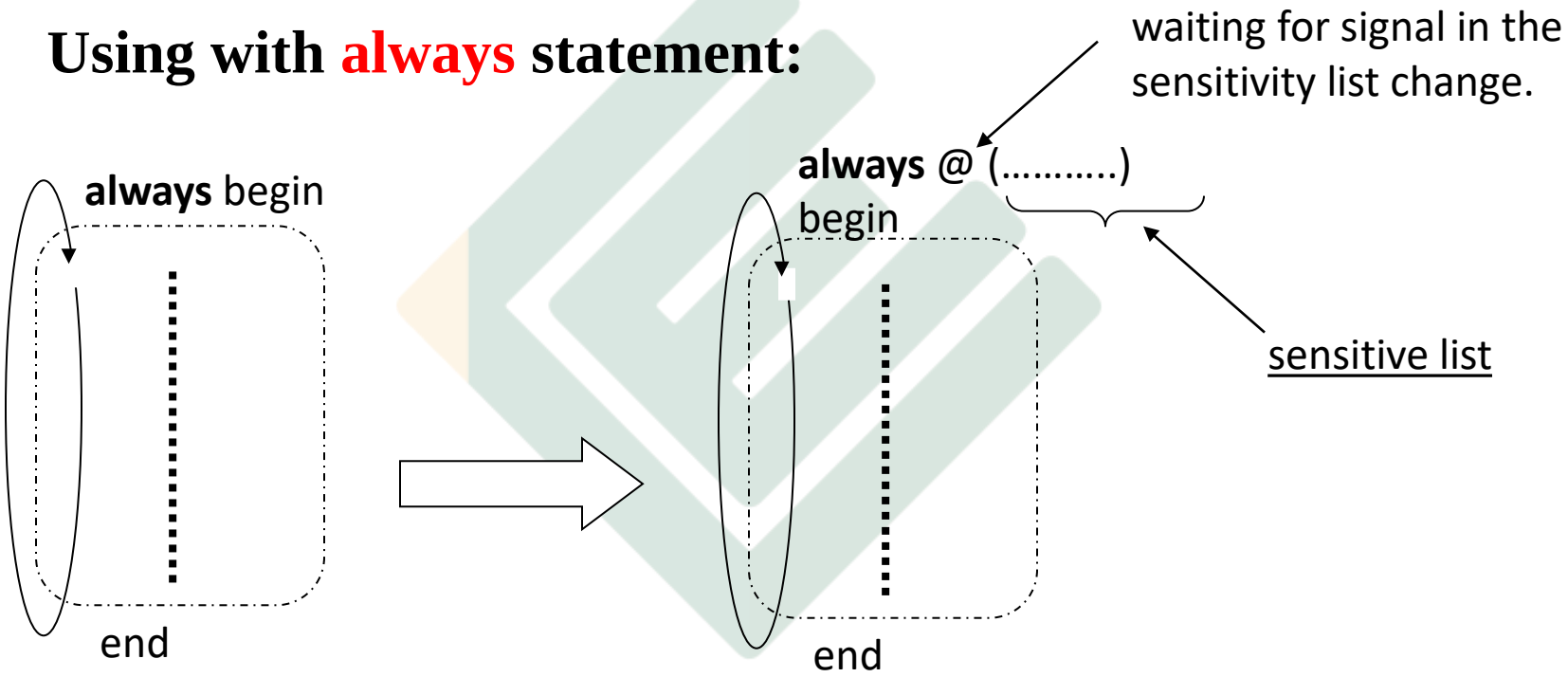
```
//delay-based timing control
#10 rega = a+b;
reg a = #10 a+b;  //intra-assignment delay

//event-based timing control
```

- @(clock) q = d; //q = d is executed whenever signal clock changes value
- @(posedge clk) q=d;
- q = @(posedge clock) d;
- @(a, b, c) y = (a & b) | (b & c) | (a & c); //equivalent @(a or b or c)
- @(*) // or @*, equivalent to @ (a or b or c or d or f)
  y = (a & b) | (c & d) | myfunction(f);

```
//wait control
wait (!enable) #10 a = b;
```

Procedural timing control

- **Using with always statement:**

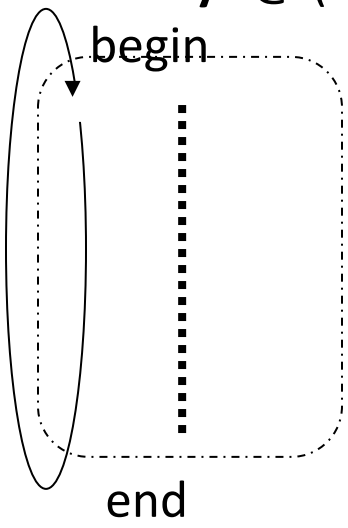


always creates an infinite loop

By adding "**@** ( )" to **always** statement: means "wait for change of signals listed in the sensitivity list" and when the change takes place, sequential block (from begin to end) is executed.

Procedural timing control

- **Using with always statement:**

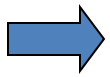


sensitive list

```
negedge: transition from 1 to x, z, or 0,
         and from x or z to 0
posedge: transition from 0 to x, z, or 1,
         and from x or z to 1
```

description	timing
always @ (posedge clk)	when signal clk rises
always @ (negedge rst_n)	when signal rst_n falls
always @ (a or b)	when signal a or b change

Do not mix single-edge (posedge / negedge) and double-edge event in an event control.



~~always @ (a or posedge clk)~~

Examples

- **4-bit counter**

```
module counter(Q , clock, clear);

output [3:0] Q;
input clock, clear;
//output defined as register
reg [3:0] Q;

always @( posedge clear or negedge clock)
begin
        if (clear)
        Q <= 4'd0; //Nonblocking assignments are recommended
                        //for creating sequential logic such as flipflops
        else
        Q <= Q + 1; // Modulo 16 is not necessary because Q is a
                                // 4-bit value and wraps around.
end
endmodule
```